

Search Pulse

A system that reads an online grocer's search log alongside its product catalogue and says exactly where search is failing shoppers — then proposes the fix without ever applying it. This document covers the concept, the data I was given, and the work I did on it.

Source data	Products indexed	Searches analysed	Bilingual names	Tests passing
828 MB	25,881	66	51,762	289

CONTENTS

01 The concept	02 The dataset	03 My workflow	04 Tech & techniques	05 What it found
----------------	----------------	----------------	----------------------	------------------

01

The concept

The problem, in one shopper

A shopper at an Egyptian online grocer typed حليب — the Arabic word for milk, with one letter too many. Search returned tea sachets, chicken cordon bleu, and a beef burger. The shop stocks **322 milk products**. None of them came back.

The shopper left. Nobody at the company knew it happened. That is the problem this project solves: **a shop cannot fix search failures it cannot see.**

What the system does

Three things, and they build on each other.

Surfaces the failures dashboard

Every query scored and ranked by *impact* — severity weighted by share of traffic — each with a plain-language diagnosis. Not a wall of numbers someone still has to interpret.

Zero-result and low-engagement queries each get their own sortable column and filter, so either failure class can be isolated directly.

When the zero-result filter comes back empty the table says *why* — on this engine that is a finding, not an absence.

Proposes the fix discovery

Synonyms and misspellings mined from the log and the bilingual catalogue, each carrying the evidence that produced it, queued for a person to approve or reject. **Nothing reaches live search on its own.**

Answers questions agent

A natural-language agent bound to eight analytics tools, plus a weekly digest of what moved and what needs attention. Every answer reports the exact tools it called.

THE DESIGN PRINCIPLE THAT SHAPED EVERYTHING

Discovery is automated; adoption is not. A wrong synonym rule degrades search for every shopper, not one. So the system proposes with a confidence score and its evidence, and a human decides. Approving only adds a rule to an export file — deploying that file stays a separate, deliberate act.

02

The dataset I was given

One file: `spinneys_database.sql` — **828 MB**, a full mysqldump of the production database behind an Egyptian online grocery. It holds **30 tables**. Seven of them carry search-quality signal; the other 23 — embeddings, stock levels, price history, Django internals — are roughly **98% of the bytes and none of the signal**.

The seven tables I used

TABLE	WHAT IT HOLDS	ROWS LOADED
<code>recommendations_searches</code>	The search log — query, the product titles returned in Arabic and English, how many, and when	66
<code>recommendations_querycount</code>	Lifetime counter per query; covers traffic older than the retained log	16
<code>catalog_product</code>	Products — SKU, URL key, default price	25,881
<code>catalog_productname</code>	Each product's name in Arabic and in English (two rows per product)	51,762
<code>catalog_category</code>	Category tree with each node's parent	272
<code>catalog_categoryname</code>	Category names in both languages	544
<code>catalog_product_categories</code>	Product-to-category links (many-to-many)	50,309

The shape of the search log

This is the most important table in the project. Its real definition from the dump:

```
CREATE TABLE `recommendations_searches` (
  `id`          bigint      NOT NULL AUTO_INCREMENT,
  `query`       varchar(255) NOT NULL,    -- what the shopper typed
  `ar_results`  longtext    NOT NULL,    -- product titles, newline-separated
  `en_results`  longtext    NOT NULL,
  `num_results` int unsigned NOT NULL,
  `created_at`  datetime(6) NOT NULL
)
```

Note what is *not* there: the results are stored as **text**, not as product foreign keys. The log says "returned the product called X", not "returned product #123". Every downstream analysis has to work from titles.

Three findings that shaped the whole design

1 — ZERO-RESULT RATE IS BLIND ON THIS DATA

Not one of the 66 searches returned nothing. The engine is embedding-based and always hands back its five nearest neighbours, so failure appears as **wrong** results, never **empty** ones. A dashboard built around zero-result rate would have scored this log a perfect 100 while it was full of failures.

2 — THERE IS NO ENGAGEMENT DATA AT ALL

No clicks, no add-to-cart, no orders. The brief asks for low-engagement queries, and they **cannot be computed directly** from this source. My answer was to measure what the shopper did *next*, and to say so everywhere the figure is reported rather than quietly presenting a proxy as the real thing.

3 — ONE WORD WEARING SIX MASKS

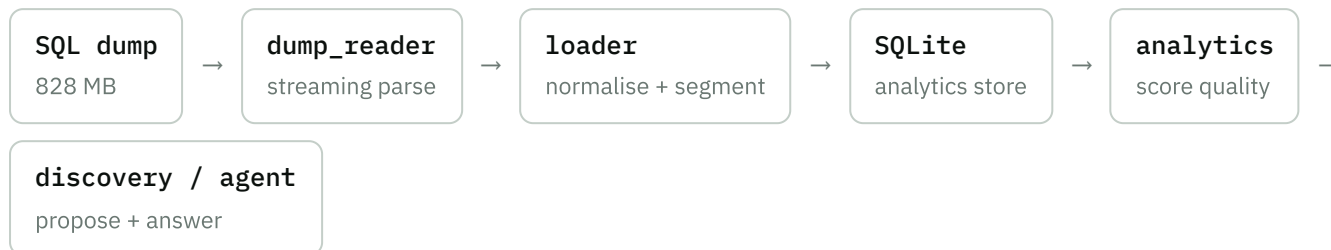
فراولة, فراولة, فراولت, فراولت, فراوله, فراول — six rows in a query report, one word in the shop. Ranked as separate queries, none looks urgent. Rolled up to the term, it is **the single largest source of failed searches in the log**.

WHAT THE DATA CANNOT SUPPORT

All 66 searches fall on 2025-01-09 within 4.5 hours, and 7.6% of them look automated. This is a QA session, not representative traffic. The rates are computed honestly but rest on a small sample — which is why I also built a mock-data generator, so the system can be exercised on 28 days of traffic with known patterns.

My workflow: 828 MB to an analytics store

The pipeline I built



One pass over the file builds every table. A full rebuild takes about 30 seconds.

Decision: read the dump directly, don't import it

- **No server to install** — and no 828 MB import to sit through.
- **No credentials to handle** — the file is read as text.
- **Flat memory** — statements are streamed, never loaded whole.
- A live-MySQL path exists alongside it (`--source mysql`) for deployments that already run the server.

ON "CHUNKING" — THE TERMINOLOGY MATTERS HERE

This is **not a RAG system**. There is no text chunking, no embedding computation, and no vector store. What the project does have is four genuine **segmentation strategies**, each solving a different problem, described below under their real names.

Worth noting: the grocer's own search engine *is* embedding-based — its table (`recommendations_productembedding`) sits right there in the dump. I skipped it deliberately, because I am measuring that engine's *output*, not rebuilding it.

The four segmentation strategies

Statement-level streaming I/O

The file is read line by line, with the parser tracking quote state so it knows where a statement actually ends — `mysqldump` can write one across several lines. Tables outside the whitelist are skipped without being parsed at all.

Rows are then buffered into **batches of 5,000** before hitting the database.

```
ingest/dump_reader.py
```

Text tokenisation NLP

Every product name is normalised, then split into tokens (anything under two characters is dropped). Each Arabic token also generates variants with its prefixes stripped (ال, بال, وال, ...لل) so فاكهة and الفاكهة are one term.

Result: **28,305 catalogue terms**, each with the number of products using it.

```
text/normalize.py
```

Sessionisation temporal

The source records no session or user id, so sessions are **derived** from timing: a gap longer than **1,800 seconds** starts a new one.

This is what makes "searched, then reworded" readable at all — without it there is no satisfaction signal. The 66 searches resolve into **3 sessions**.

```
ingest/loader.py
```

Term rollup aggregation

Every spelling of a word folds onto one row. This is not a display convenience — it is what turns six low-priority rows into one high-priority problem.

Each term gets a **verdict**: spelling, retrieval gap, assortment gap, partial query, or relevance.

```
analytics/terms.py
```

Arabic normalisation, in order

The most important function in the project: every comparison between shopper text and catalogue text passes through it. The same function runs on product names at load time and on queries at query time, so the comparison is always fair.

1. Unicode NFKC normalisation
2. Strip diacritics (harakat, sukun, shadda)
3. Remove tatweel (.)
4. Fold letters: ا → آ | إ | ئ · ي → ى · ه → ة · و → ؤ · ي → ئ
5. Fold Arabic-Indic digits to ASCII
6. Case-fold Latin text
7. Punctuation to spaces, collapse whitespace

THE JUDGEMENT BEHIND IT

Normalisation must collapse the differences that carry **no meaning** and preserve the ones that do. فراولة and فراوله become one term — correct. But حليب and حليين stay distinct — which is what keeps the misspelling *measurable* instead of silently absorbed.

Integrity, not assumption

The bulk load runs with foreign keys disabled for speed, then deletes orphaned rows (a product referenced by a category link but missing from the product table), then **actually verifies** that no violation remains. If one does, the load fails rather than publishing an inconsistent store.

04

Tech stack and techniques

LAYER	CHOICE	WHY THIS ONE
Language	Python 3.11+	Mature text and analysis libraries; one language for ETL and API
API	FastAPI + Uvicorn	Interactive docs at <code>/docs</code> for free, input validation built in
Storage	SQLite (WAL)	One file, no server, whole store rebuilds from one command
Frontend	HTML + CSS + JS	No framework, no build step — the fewest moving parts that can break
Agent	Anthropic SDK	Manual tool loop so every answer returns its own call trace
Testing	pytest	289 tests, under 11 seconds, no network and no API key
Linting	ruff	Style and common-error checks
Reporting	CSV + JSON	Flat tables for Power BI

The techniques I applied

Edit distance Damerau-Levenshtein

For misspelling detection, with a budget **proportional to word length**: 3 characters or fewer → zero edits, up to 6 → one edit, longer → two.

Why: two edits turn `pas` into `pcs`, an unrelated word — but the same two edits merely nudge a long word to its correct spelling.

Bilingual lexicon induction

Dice coefficient

Every product carries an Arabic and an English name. Mining co-occurrence across 25,881 products induces a translation lexicon **with no dictionary**: حليب ↔ milk on 293 products.

Two terms that translate to the same English word are synonyms of each other — which is how حليب ≡ لبن is established by evidence rather than assumed.

Composite severity 5 signals

Each query scores 0–1 from five independent signals, weighted: lexical miss 0.30 · dissatisfaction 0.25 · incoherence 0.20 · under-fill 0.15 · instability 0.10.

The careful part: if a signal could not be measured for a given query it is dropped and the remaining weights are renormalised — so a query with no coherence reading is never silently credited with perfect coherence.

Tool-constrained agent tool use

The agent is bound to **eight tools** and answers only through them. It never writes SQL and never sees the database, so it cannot invent a metric — the worst it can do is pick the wrong tool, which the trace shows.

It works with no API key: a deterministic planner routes the question by intent and reads the *same* tools. Plainer wording, identical numbers.

Behavioural proxies, since engagement is absent

- **Followed by a repeat or rewording** within 300 seconds → the result set did not satisfy.
- **Nothing followed** → ambiguous, and deliberately not counted. A session ending might mean the shopper found it and checked out, or gave up. The log cannot tell those apart.
- **Re-fired within 2 seconds with identical results** → not a human reaction. Flagged as automated and excluded from engagement, not deleted — which queries attract bot traffic is itself worth knowing.

The commands

```
searchiq etl                # build the store from the dump (~30s)
searchiq sample-data        # or: generate mock data if no dump
searchiq report              # search-quality summary in the terminal
searchiq discover            # find synonyms and misspellings
searchiq suggestions         # read the review queue
searchiq approve 3           # record a decision
searchiq export -o rules.json
searchiq ask "which queries are wasting the most traffic?" --trace
searchiq digest --days 7
searchiq export-bi           # flat tables for Power BI
searchiq serve               # dashboard on 127.0.0.1:8000
```

05

What it found in the real data

These are outputs of the shipped code against the real dump, not illustrations.

Overall health: 64.4 / 100

METRIC	VALUE	WHAT IT MEANS
Irrelevant results	12.1%	Search returned nothing mentioning the query
Zero results	0.0%	The blind metric — the engine always returns something

METRIC	VALUE	WHAT IT MEANS
Under-filled	6.1%	Fewer than 5 products came back
Retrieval gaps	4	Stocked in the catalogue, not surfaced by search
Retried or reworded	88.5%	Behavioural proxy, not click data
Machine-like traffic	7.6%	Repeats under 2 seconds — excluded from engagement

The three worst cases

حليب — 3 searches · severity 0.59

The shopper typed "milk" with one letter too many. Search returned tea with milk, cordon bleu, and a beef burger. The catalogue holds **322 products** using حليب — none came back.

This is not an assortment problem, it is a retrieval fault.

فاراولت — 2 searches · severity 0.84

A misspelling of "strawberry". It returned exactly one product: Agnesi Farfalle Pasta — pasta, not strawberry, but lexically close. The catalogue holds **308 strawberry products**.

فاكهة — 11 searches · severity 0.40

A perfectly spelled word meaning "fruit". Search returned a kitchen knife, a serving platter, a guava, and a bag of sugar. All 11 searches were flagged, and 100% of them were followed by another search. **Not spelling — relevance.**

What automated discovery proposed

KIND	PROPOSAL	CONFIDENCE	EVIDENCE
Misspelling	فاراوله ← فراوله	0.95	1 edit · 308 products · shopper retyped it themselves
Misspelling	حليب ← حلين	0.90	1 edit · 322 products
Partial query	past → pasta	0.95	Prefix of a term used by 384 products
Synonym	حليب ≡ لبن	0.74	Rewording + result overlap + cross-language alignment
Synonym	milk ≡ حليب	0.58	Cross-language catalogue alignment

MISSPELLING VS. PARTIAL QUERY — A DISTINCTION THAT MATTERS

past → pasta is **not** a misspelling — it is a shopper who had not finished typing. Correcting their query would change their intent. The rule I used: if the weaker term is a *prefix* of the stronger one, it is a partial query and the fix is autocomplete. If the weaker term is *longer*, the shopper typed a character too many and the fix is a spelling dictionary.

CORRECTION DIRECTION COMES FROM THE CATALOGUE, NEVER FROM FREQUENCY

حليب is correct because **322 products** use it — regardless of which spelling was typed more often. Had I ranked by frequency, a small sample with a repeated typo could flip the direction and make the system "correct" the right spelling into the wrong one.

Also delivered

- **Weekly digest** — period-over-period performance, searches that came back empty split into retrieval gaps and assortment gaps, and everything pending review. Available from the CLI, the API, and as a Markdown download.
- **Mock data generator** — writes a real mysqldump file with planted zero-result queries, misspellings, partial queries and low-engagement terms, so the system runs out of the box without the 828 MB source.
- **Power BI export** — six flat tables over CSV and JSON. Every figure is read from the same functions the dashboard uses, so a report cannot disagree with the screen.

Every figure in this document is an output of the shipped code against `spinneys_database.sql` — 289 tests passing, store built from the real data.